Estrategia de calidad — ContaStock Pro

Juan Carlos Díaz Motta · QA Senior · Prueba técnica HelpPeople

1. Enfoque y criterio de cobertura

ContaStock gestiona inventario, así que el mayor riesgo de negocio es la **integridad del stock**: un dato de inventario incorrecto produce compras equivocadas, faltantes y pérdida de auditoría. Por eso la cobertura se priorizó por **riesgo = impacto × probabilidad**, no por número de pantallas. El flujo de inventario (productos y movimientos) concentra la automatización; categorías y órdenes, con menor impacto, se cubren con revisión manual. Toda aserción verifica **persistencia** (recargar y volver a consultar), no solo el mensaje de éxito.

2. Matriz de riesgo, caso, prioridad y nivel

Riesgo	Imp.	Prob.	Prioridad	Caso	Nivel de prueba
Salida deja stock negativo	Alto	Media	**P1**	QA02	API/servicio + UI E2E (smoke)
Eliminar producto con movimientos	Alto	Media	**P1**	QA02	API + UI E2E
Alta de producto no persiste	Alto	Baja	**P1**	QA03	UI E2E (smoke)
XSS en campos de texto	Alto	Media	**P1**	DEF-13	Seguridad (SAST/DAST) + manual
Entrada/ajuste calcula mal el stock	Alto	Baja	P2	QA05	Unitaria + API + UI E2E
KPI crítico incorrecto	Medio	Media	P2	QA01	Unitaria + UI E2E
Validaciones de formulario	Medio	Alta	P2	QA04	Componente + UI E2E
Búsqueda y filtros	Medio	Media	P2	QA06	API + UI E2E
Fecha futura / edición múltiple	Medio	Media	P3	DEF-08/11	UI E2E (backlog)
Textos, títulos, mayúsculas	Bajo	Alta	P4	DEF-01/06	Revisión manual / visual

UI, API o manual. En **UI** dejaría solo lo que el usuario debe ver extremo a extremo: navegación, alta, mensajes de rechazo y persistencia. Las reglas de cálculo (stock antes/después, conteo crítico) y la validación del servidor van en **API y unitarias**, porque son más rápidas y estables; aquí la app no tiene backend, así que esa capa queda como recomendación. En **manual/exploratorio** quedan la usabilidad, los textos, la accesibilidad, la seguridad básica y los flujos nuevos antes de automatizarlos.

3. Qué automatizaría después y qué dejo fuera

Siguiente iteración: DEF-08 (fecha futura), DEF-11 (edición múltiple), CRUD de categorías y órdenes, pruebas de API cuando exista backend, accesibilidad automatizada con axe-core en las pantallas principales y regresión visual de los modales. **Fuera de alcance de E2E:** textos y ortografía (revisión visual), rendimiento (k6, en otra capa), compatibilidad con más navegadores (el enunciado pide Chromium) y seguridad profunda (pipeline SAST/DAST).

Condiciones que bloquean una liberación:

1. Cualquier fallo de un caso P1 o de la suite smoke.
2. Un defecto crítico o mayor abierto en inventario o seguridad sin aceptación formal del riesgo.
3. Una prueba P1 intermitente sin causa raíz conocida.
4. Un pipeline sin evidencias (reporte o trazas faltantes) o un ambiente distinto al build esperado.

4. Cómo investigo una prueba intermitente

1. **Medir:** `--repeat-each=20` y `--retries=0` para obtener la tasa real de fallo; comparar headless/headed y 1 vs. N workers.
2. **Evidencia:** comparar la traza de Playwright de una ejecución que falla con una que pasa (red, consola, DOM, tiempos).
3. **Clasificar:** ¿**producto** (condición de carrera real), **automatización** (localizador o espera frágil, datos compartidos) o **ambiente** (servicio lento, datos sucios)?
4. **Contener:** si no es del producto, marcar con `test.fixme` + ticket + responsable + fecha límite (cuarentena visible, nunca un `retry` silencioso). Si es del producto, abrir el defecto con la traza.
5. **Prevenir:** la causa raíz se lleva a una guía de buenas prácticas y a una regla de lint (p. ej. `no-wait-for-timeout`).

5. Caso práctico: "la versión sale hoy, el flujo principal falla intermitente"

Evidencia que buscaría (en ~1 hora):

- La tasa de fallo con `--repeat-each` en el build candidato vs. la versión en producción.
- Trazas, consola y red de las ejecuciones fallidas; si el fallo coincide con un paso concreto.
- Logs del backend correlacionados por hora.

- Commits incluidos en el release que tocan ese flujo.
- Si la falla ocurre también por API (descarta la UI) y si depende de datos o del ambiente.

Acciones: reproducir en pareja con desarrollo usando la traza (no "en mi máquina"). Aislar si es producto, prueba o ambiente. Si es la prueba, corregirla o ponerla en cuarentena con ticket y validar el flujo manualmente con evidencia. Si es el producto, estimar el impacto y buscar mitigaciones: feature flag, rollback, despliegue gradual.

Recomendación: no liberar el flujo principal con una falla intermitente sin causa explicada. Una intermitencia en el flujo principal suele ser una condición de carrera real que en producción, con más carga, falla más, no menos. Opciones en orden:

- (a) Si se demuestra que es la prueba o el ambiente, liberar con validación manual documentada.
- (b) Liberar con la funcionalidad detrás de un flag, o con despliegue canario y monitoreo y rollback listo.
- (c) Si negocio decide continuar con el riesgo, que sea una **aceptación de riesgo explícita y firmada** por el dueño del producto.

QA recomienda e informa; **la decisión de negocio es del Product Owner**, no de QA.

Responsables: Product Owner (decide), líder técnico y desarrollador del cambio (causa raíz), DevOps/SRE (monitoreo y rollback), soporte (aviso temprano a usuarios). **Comunicación:** un mensaje corto y escrito al canal del release con: qué falla, frecuencia medida, impacto en el usuario, lo que sabemos y lo que no, opciones con su riesgo, recomendación y hora de la próxima actualización. Queda trazado en el ticket del release.

6. Mi primer mes como QA (renovación del proceso)

- **Semana 1, diagnóstico:** mapa de productos y riesgos, revisión de bugs escapados de los últimos 3 meses, pipeline actual y *Definition of Done*.
- **Semana 2, base:** framework estándar (este proyecto como plantilla), convenciones, revisión de código de pruebas en cada PR (checklist: localizadores, datos, aserciones, sin esperas fijas) y smoke en CI.
- **Semana 3, shift-left:** criterios de aceptación verificables en el refinamiento (sesiones de "3 amigos"), matriz de riesgo por épica y regresión priorizada.
- **Semana 4, IA con control:** asistentes de IA para borradores de casos desde historias, triage de fallos sobre trazas y generación de datos. Siempre con revisión humana, sin datos sensibles, y con lo aprendido documentado como guía del equipo. Mantenimiento: presupuesto fijo (~20 %) para refactor y cuarentena de flaky.
- **Indicadores:** (1) **defectos escapados a producción** por release; (2) **tasa de pruebas intermitentes** (meta < 2 %) y tiempo de feedback del pipeline (meta < 10 min); (3) **% de riesgos P1 con prueba automatizada**.